01 / ROLE AND ROADMAP

Puneet's Phase 1 work guide

PROJECT 11 / AUTOMATIC CLOTHING CATEGORY RECOGNITION

Inventory Image Tagging via Handcrafted Features and a Small CNN
Phase 1 focus: classical machine learning on Fashion-MNIST.

Your role: Feature engineering and baseline lead

Build one reliable pixel transform and one fixed HOG transform.

The five-person division

Member	Primary ownership
Nakul	Integration, literature and report Repository, master notebook, report assembly
Sudeshhna	Data quality and EDA Loader, fixed split, audit, EDA figures
Puneet	Features, Logistic Regression and kNN Pixel/HOG features and four model rows
Samarth	Random Forest, SVM and run harness Four model rows and experiment logs
Neelanshu	Evaluation and error analysis Metrics, timing, complexity, error figures

How to use this guide

Read pages 2-4 first, then work through your practical pages in order. Pages 13-16 explain handoffs, viva preparation, the proposed schedule and sources. Your guide is a work plan and learning manual, not a completed experiment or a submission-ready report.

All five members must understand the entire pipeline. Ownership means you implement and explain an area; it does not mean the others can skip learning it. Each member also writes their report section, prepares their slides, and reviews another member's work.

Prepared 27 October 2026. All workload estimates, code organization, hyperparameter grids and internal milestones are recommendations, not faculty rules. No model scores have been invented.

02 / SHARED FOUNDATIONS

What Phase 1 actually requires

The supplied CSAI instructions are the course requirements. Your project description supplies the technical problem. This guide adds a practical way to satisfy both; it does not turn its suggestions into extra faculty requirements.

Course requirement	Evidence the team should prepare
Literature review	A cited comparison of relevant sources and the gap your experiments investigate.
Dataset quality and EDA	Shape/range/label checks, class counts, sample images and a written audit.
Feature engineering	Normalized flattened pixels and a justified HOG configuration.
Theoretical rigor	Explain each model, its assumptions, tuning choices and limitations.
Model application	Two representations with Logistic Regression, kNN, Random Forest and SVM.
GitHub and code quality	Reproducible code, clear README, saved settings, outputs and contribution evidence.
Presentation and individual viva	Every member presents their work and explains the shared experiment.
Project report	A coherent report in LaTeX/IEEE format, using the course template if supplied.

Scope boundary

Complete the classical ML component now. Save the CNN, pooling, dropout, CNN early stopping, AML-versus-CNN experiments and any implemented hybrid approach for Phase 2. A brief future-work paragraph is enough at this stage. The Phase 1 comparison is pixels versus HOG across four classical classifiers.

The attachment gives no Phase 1 evaluation date, page limit, presentation duration, grading weights, minimum accuracy, or required number of papers. Its September dates concern project/team selection. Check the course channel for later evaluation announcements; do not treat the proposed schedule here as an official deadline.

03 / SHARED FOUNDATIONS

Machine learning from zero

Here, **AML** means the classical/applied machine-learning part described in your project. You do not hand-write a rule saying "sleeves mean shirt." Instead, you show a model labeled examples so it can learn a rule from image measurements.

Term	Plain meaning in this project
Sample; label	One clothing image; its correct category number. The label is the answer.
Feature; representation	A numeric input. One image becomes either 784 pixel values or a HOG vector.
X and y	X is the feature matrix: one row per image. y is the matching list of labels.
Model; fit; predict	A learned decision rule; learn from X and y; assign labels to new X.
Parameter	Something learned during fitting, such as a Logistic Regression coefficient.
Hyperparameter	A setting chosen before fitting, such as k in kNN or C in an SVM.
Training / validation / test	Practice examples / development exam / final independent exam.
Overfitting	Doing very well on training data but failing to generalize to held-out data.
EDA	Exploratory data analysis: inspect the data before deciding how to model it.
Reproducibility	Another person can repeat the process using the same data, code and settings.

Walk through a single image

A grayscale image is a small grid of brightness values. Divide its raw pixel values by 255 to put them on a 0-1 scale. Flatten the grid to a vector, or compute a HOG description of local edge directions. A fitted classifier then maps that vector to one of the category IDs. Map the ID to the correct name for display.

Your first learning exercise

Before your assigned work, use a notebook to print an array's shape, show one image, print its label, and call fit then predict on a tiny example. Explain aloud why a row of X must stay attached to the same row of y. If that connection is broken, model training becomes meaningless.

Important nuance: flattening does not erase pixel values or their recoverable ordering. It gives a conventional classifier no built-in local-neighborhood or translation structure. HOG encodes local geometry, but its feature rule is fixed before classifier training.

04 / SHARED FOUNDATIONS

One shared, fair experiment

Freeze this protocol before comparing models

- Keep the official training and test partitions separate. Create a stratified 80/20 split of the official training partition with random_state=42: 48,000 fit images and 12,000 validation images. Save the indices once; everyone reuses them.
- Use two fixed representations and four model families: pixels/HOG × Logistic Regression/kNN/Random Forest/SVM. This produces eight main comparison rows. Add one DummyClassifier reference row outside those eight.
- Train on the fit partition, choose settings using validation macro F1, and use validation accuracy as a tie-breaker. If still tied, choose the simpler or faster option using validation measurements. Log the rule before seeing test scores.
- Fit learned scalers only on fit data. A fixed divide-by-255 transform and fixed per-image HOG extraction do not learn dataset statistics. Use a Pipeline for StandardScaler plus the classifier.
- After freezing one configuration per row and naming the validation-selected overall model, evaluate the eight frozen models on the official test partition. Do not choose new hyperparameters or a new overall winner from test results.

Training size and compute policy

Default: all eight final models use the same 48,000-image fit partition and the same 12,000-image validation partition. Keep those fitted models for final testing; do not silently refit some models on all 60,000 training images. This preserves the same split for Phase 2.

If compute makes this infeasible, agree before final runs on one saved stratified 12,000-image subset drawn only from the 48,000 fit images. Use that identical subset for all eight rows, retain the same validation/test partitions, and label the table "12,000 fit images per model." Exploratory subset runs must never masquerade as full-size results.

Required result package

Report accuracy, macro F1, recall for all ten classes, training time, inference time with a stated protocol, and a meaningful model-complexity description. Explain when a classical model has no neural-network-style parameter count. Save all eight rows, including weak runs; discuss shirt/T-shirt/pullover confusion using observed examples.

Protocol choices on this page are recommendations for consistency. The official split and the stated project metrics come from your project description; the specific validation fraction, seed and selection rule are proposed here.

05 / YOUR WORK

Your ownership and first session

You decide how the image becomes numbers a classical model can use, then implement Logistic Regression and kNN on both representations. The representation is not an incidental preprocessing step: it is half of the main research question.

What you personally build

- **src/features.py:** raw-to-0-1 conversion, flattened pixels, fixed HOG extraction and feature metadata. Preserve image order and original IDs.
- **notebooks/02_features_lr_knn.ipynb:** feature demonstrations, small smoke runs, validation tuning and four final rows: pixels+LR, HOG+LR, pixels+kNN, HOG+kNN.
- **Reusable model factories:** coordinate src/models.py with Samarth so you do not overwrite each other's work. Save full preprocessing/classifier Pipelines where a scaler is learned.
- **Report evidence:** a HOG visualization, feature-dimension table, justified LR/kNN settings, observed results and limitations. Write your own methods/results text and slides.

First 90 minutes

0-20 min: inspect Sudeshhna's raw image and label contract. **20-45 min:** normalize and flatten one image, then print its shape and range. **45-65 min:** compute one HOG vector and display its visualization. **65-90 min:** run a tiny Logistic Regression example and explain the difference between fit and predict.

Your model rows must be comparable

Use the team's saved fit and validation IDs, not a private train_test_split call. Train the two representations on the same image IDs. A result difference should come from representation/model choices, not a different set of easy examples.

Your handoff is accepted when

Samarth can import your feature function and reproduce feature rows for several agreed original IDs. Neelanshu can use your saved metadata and Pipeline to reproduce validation predictions. Every artifact states feature name, dimension, HOG settings, scaler and training size.

Suggested effort: 20-26 hours including feature coding, theory, four model comparisons, result handoff, writing and viva preparation.

06 / YOUR WORK

Pixels: build a reliable reference

Raw pixels are a useful baseline because they require few design choices. A 28 × 28 image has 784 intensities. Reshaping it into a 784-value row preserves values in a fixed order, but a standard classifier has no built-in knowledge that adjacent entries came from a local image neighborhood.

```
import numpy as np

def normalized_images(raw):
    raw = np.asarray(raw)
    if raw.ndim != 3 or raw.shape[1:] != (28, 28):
        raise ValueError("Expected shape (N, 28, 28)")
    if raw.dtype != np.uint8:
        raise TypeError("Expected raw uint8, not scaled images")
    return raw.astype(np.float32) / 255.0

def pixel_features(raw):
    images01 = normalized_images(raw)
    return images01.reshape(len(images01), 784)

Xp = pixel_features(raw_images)
assert Xp.shape == (len(raw_images), 784)
assert np.isfinite(Xp).all()
assert Xp.min() >= 0 and Xp.max() <= 1
```

Know two different uses of "scaling"

Divide by 255: a fixed known conversion of raw brightness values, using no sample statistics.
StandardScaler: learns a mean and standard deviation per feature from fit data, then applies those same values to validation/test data. The second operation must be fitted only inside the training process. [R5]

Proposed preprocessing per classifier

Classifier	Pixels	HOG
Logistic Regression	StandardScaler inside Pipeline	StandardScaler inside Pipeline
kNN	Use 0-1 features directly	StandardScaler inside Pipeline
Random Forest	Use 0-1 features directly	Use HOG directly
RBF/linear SVM	StandardScaler inside Pipeline	StandardScaler inside Pipeline

This is a defensible starting protocol, not a claim that one scaler is optimal for every representation. Per-pixel standardization can magnify low-variance background pixels; direct 0-1 pixels give kNN a simple shared intensity scale. Document any alternate scaler experiment as a validation-only choice.

Never scale labels. Label numbers are category IDs, not measurements. Do not divide y by 255, one-hot encode it unnecessarily, or concatenate it into X.

07 / YOUR WORK

HOG: understand and implement it

Histogram of Oriented Gradients describes local changes in brightness. It computes image gradients, groups their directions within small cells, and normalizes groups of cells. The idea is to describe edge layout rather than remember every raw intensity. Its feature rule is handcrafted; only the later classifier learns from category labels. [R3-R4]

Fix one configuration first

```
from skimage.feature import hog

HOG_SETTINGS = dict(
    orientations=9, pixels_per_cell=(4, 4),
    cells_per_block=(2, 2), block_norm="L2-Hys",
    transform_sqrt=False, feature_vector=True, channel_axis=None)

def hog_features(raw):
    images01 = normalized_images(raw)
    rows = [hog(im, **HOG_SETTINGS) for im in images01]
    return np.asarray(rows, dtype=np.float32)

Xh = hog_features(raw_images)
assert Xh.shape == (len(raw_images), 1296)
assert np.isfinite(Xh).all()
```

Explain every setting

9 orientations: nine direction bins. **4 × 4 pixels per cell:** small local regions. **2 × 2 cells per block:** neighboring cells normalized together. **L2-Hys:** the chosen block-normalization rule. **channel_axis=None:** this is grayscale. A consistent upright input is required; HOG is not automatically rotation invariant.

Derive the length instead of guessing

28/4 gives 7 cells per axis. Sliding a 2-cell block by one cell gives 6 blocks per axis. Each block contains 2 × 2 cells × 9 bins = 36 numbers. Total feature length is 6 × 6 × 36 = **1,296**. This is larger than 784, so HOG here is a structural representation, not dimensionality reduction.

Optional validation experiment

If time remains after the eight core rows, test pixels_per_cell=(7,7) with the other settings fixed: 4 cells per axis, 3 blocks per axis and 324 features. Treat it as a separate labeled feature configuration. Do not select a HOG setting after viewing test scores or compare models using undocumented mixtures of settings.

Keep the core HOG setting fixed across all four classifiers for the cleanest representation comparison. Optional configuration experiments belong in a clearly separate development table.

08 / YOUR WORK

Validate features and manage caches

A feature cache saves compute, not responsibility

Computing a fixed HOG vector independently for each image can be cached without learning from validation labels. A learned scaler, PCA or supervised feature selector cannot be fitted on all cached rows. Keep unscaled feature matrices separate from fitted Pipeline artifacts.

- For every cache save: source partition, original sample IDs in row order, feature name/configuration, feature dimension, dtype, source-data fingerprint and code/package versions.
- Name caches clearly, such as hog4_fit_full.npy and hog4_val.npy. Pair them with metadata and ID arrays. Do not reuse a filename after changing the HOG configuration.
- Use float32 unless a library requires another type. A 48,000 × 1,296 float32 matrix alone is about 237 MiB; extra copies, scalers and parallel jobs require more memory.
- Cache only after confirming a few examples by recomputing them individually. A cache with the right shape can still be stale or misordered.

Show what HOG sees

```
from skimage import exposure
one = normalized_images(raw_images[:1])[0]
vec, picture = hog(one, visualize=True, **HOG_SETTINGS)
picture = exposure.rescale_intensity(picture, in_range="image")
fig, ax = plt.subplots(1, 2, figsize=(6, 3))
ax[0].imshow(one, cmap="gray", vmin=0, vmax=1)
ax[0].set_title("Original fit image")
ax[1].imshow(picture, cmap="gray")
ax[1].set_title("HOG visualization")
for a in ax: a.axis("off")
fig.tight_layout()
fig.savefig("figures/hog_example.png", dpi=200)
```

The HOG visualization illustrates edge-direction structure. It is not the feature vector itself, an explanation of what a classifier learned, or a reconstructed original image. Use an actual fit image and include its ID in the caption.

Feature acceptance checklist

Check expected dimension, finite values, correct row count, deterministic output for the same raw image, and exact correspondence of feature rows to image IDs. Check orientation on the input image. Pass a small batch and a one-image batch through the same function; outputs should agree for the matching image.

Imports in the visualization snippet also require matplotlib.pyplot as plt. Create figures/ before saving. These are small recipes to place in your notebook, not standalone scripts.

09 / YOUR WORK

Logistic Regression from intuition to code

Despite its name, Logistic Regression is a classifier here. For class c it learns a score z_c = w_c · x + b_c. Softmax converts the ten scores into relative probabilities, and the largest determines the predicted category. Training minimizes multiclass cross-entropy plus regularization. [R6]

What the key ideas mean

Cross-entropy: wrong confident predictions receive a larger penalty. **Regularization:** discourages overly large weights. **C:** inverse regularization strength; smaller C means stronger regularization. **Linear:** the score is linear in the supplied feature vector; HOG can still change what information that linear rule can use.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

def make_lr(C=1.0):
    return Pipeline([
        ("scale", StandardScaler()),
        ("model", LogisticRegression(
            C=C, solver="lbfgs", max_iter=1000,
            random_state=42))])

lr = make_lr(C=1.0)
lr.fit(Xtr, ytr)
val_pred = lr.predict(Xva)
score = f1_score(yva, val_pred, average="macro",
                 labels=np.arange(10), zero_division=0)
print("validation macro F1:", score)
```

Your small tuning plan

Run C=1 first on the smoke data for both features. If that works, try a compact validation-only grid C in {0.1, 1, 10}, giving three configurations per representation. Use the shared selection rule. Preserve validation scores and convergence warnings for every attempted configuration.

Convergence warning is evidence

If optimization hits max_iter, check input finiteness, feature scaling and the warning. Increasing max_iter may be appropriate; silently ignoring warnings is not. Record that rerun and its settings. Do not change the solver, regularization and feature configuration simultaneously, because you will not know which change mattered.

Explain complexity

For the standard ten-class linear coefficient form, count model.coef_.size + model.intercept_.size. This is 10(d+1): 7,850 scalars for d=784 or 12,970 for d=1,296. Report actual array shapes from the fitted model; the scaler has additional stored statistics. These are counts, not accuracy predictions.

10 / YOUR WORK

kNN from intuition to code

k nearest neighbors stores training feature vectors. For a new vector it finds the k closest stored examples and predicts from their labels. It learns no LR-style coefficient vector. Most of the work happens during prediction, so a fast fit does not imply fast inference. [R7]

Understand distance and voting

Euclidean distance is sqrt(sum_j (x_j - x'_j)^2). A feature with a much larger scale can dominate this distance, which is why representation and scaling matter. Uniform voting gives every neighbor one vote; distance weighting gives closer examples more influence. Small k may be sensitive to noise, while large k can blur class boundaries.

```
from sklearn.neighbors import KNeighborsClassifier

def make_knn(feature_name, k=5, weights="distance"):
    scale = StandardScaler() if feature_name == "hog" else "passthrough"
    return Pipeline([
        ("scale", scale),
        ("model", KNeighborsClassifier(
            n_neighbors=k, weights=weights,
            metric="minkowski", p=2,
            algorithm="brute", n_jobs=1))])

knn = make_knn("hog", k=5, weights="distance")
knn.fit(Xtr, ytr)
val_pred = knn.predict(Xva)
```

Your small tuning plan

Start with k=5 and distance weighting. If time permits, test k in {3, 5, 7} and weights in {uniform, distance}: at most six settings per representation. Use identical validation IDs, consistent threading and the same metric. Record total prediction time, since evaluating many kNN candidates can be costly.

Why force brute-force search here?

These feature vectors have hundreds of dimensions. A simple explicit brute-force method makes the chosen search approach clear and reproducible. It is not a promise of optimal speed. If you investigate another search algorithm, document it as a validation-stage implementation choice, then benchmark consistently.

What to inspect after fitting

For a few validation errors, retrieve nearest training neighbors and display their labels with the queried image. This can explain why the representation groups two clothing categories together. The visual impression remains a hypothesis unless supported across more cases; avoid claiming that one neighbor panel proves a general cause.

For complexity, report stored training rows, feature dimension, approximate feature-memory bytes and serialized file size. Learned coefficient count is "not applicable: instance-based classifier." Never write "zero parameters, therefore free prediction."

11 / YOUR WORK

Run your four rows and export evidence

Execute in a controlled order

- 1. Load the shared split and compute unscaled pixel/HOG features for the same fit/validation IDs. Confirm shapes and row order.
- 2. Run LR and kNN on the common smoke subset to catch interface problems. Mark these artifacts smoke and keep them out of final tables.
- 3. Apply the agreed compact tuning plan on the agreed development profile. Log every configuration, its training size, warnings and validation metrics.
- 4. Freeze one LR configuration and one kNN configuration per representation using the shared selection rule. Ensure all four final rows use the same fit size as Samarth's rows.
- 5. Save fitted pipelines and feature metadata. Export validation predictions with original IDs for Neelanshu. Do not evaluate the official test set privately.

Save enough to reproduce one prediction

```
import joblib
bundle = {
    "pipeline": fitted_pipe,
    "feature_name": feature_name,
    "hog_settings": HOG_SETTINGS if feature_name == "hog" else None,
    "raw_input": "uint8_N_28_28",
    "train_n": len(ytr),
    "split_hash": split_hash,
    "run_id": run_id,
}
joblib.dump(bundle, f"artifacts/{run_id}.joblib")
```

The receiving code must compute features according to feature_name before calling pipeline.predict. Confirm reload-and-predict matches the original output on a few validation examples. joblib files should be loaded only from your trusted team artifacts, because deserializing untrusted Python objects can execute code.

Write results without overclaiming

Compare HOG versus pixels within LR and within kNN. Give changes in percentage points for accuracy and F1 only if the scales are stated consistently. Ask whether any aggregate gain comes with worse recall for a confusing class. Discuss prediction speed and memory alongside correctness.

A useful sentence template is: "Under the same split and training size, [representation] changed validation macro F1 by [measured amount] for [classifier]; the main observed recall differences were [classes]." Replace every bracket using actual saved results or omit the sentence.

12 / YOUR WORK

Troubleshooting and your completion test

Symptom	Diagnosis and response
HOG length differs	Wrong cell/block settings, image size or channel handling. Print the saved feature configuration and verify one image.
Features look tiny or images black	Check double normalization and input dtype. Your public feature functions expect raw uint8 images.
All predictions are one class	Check y alignment, label counts, features with zero variance and whether fit actually ran on the intended data.
LR convergence warning	Check finite inputs and scaling, then inspect iteration limit/settings. Record the warning and any justified rerun.
kNN validation takes a long time	Expected prediction cost can dominate. Batch queries, avoid nested parallel jobs, and reduce optional search trials.
Saved model fails after reload	Missing feature configuration, incompatible versions or a different input shape. Reload in the same pinned environment.
Unexpectedly perfect validation	Audit overlap, target leakage, row order and whether validation was accidentally used in fitting.

Your final checklist

- Both feature functions preserve image order and accept the documented raw input. Expected dimensions are verified and metadata is saved.
- Pixel and HOG models use appropriate, documented preprocessing. No scaler is fitted on validation or test data.
- You have four final comparison rows with consistent train_n and split hash, complete run logs and preserved validation predictions.
- Model artifacts reproduce predictions after reloading, and Neelanshu has the input transformation needed for end-to-end timing.
- Your methods text explains flattening, HOG, LR, kNN, hyperparameters, validation selection and memory/inference trade-offs.

Explain these concepts without notes

Why is HOG length 1,296 in this setup? Why is HOG not a learned representation? What changes when C decreases? Why does kNN predict slowly? Why is "flattening destroys all spatial information" too strong? How does a Pipeline prevent scaler leakage?

Your concrete completion standard

A teammate can reproduce your four rows and trace the same raw image through either representation into a fitted LR or kNN prediction.

13 / TEAM DELIVERY

Handoffs and shared file contract

Each person works on their own files, commits meaningful changes, and opens a review request or asks a teammate to review. Share generated evidence with file paths and the exact run ID; a screenshot in a group chat is insufficient.

Sender → receiver	What is handed over; acceptance check
Sudeshhna → everyone	Loader, labels, split_indices.npz, audit.md. Row counts and train/validation IDs agree.
Puneet → Samarth + Neelanshu	Feature functions, HOG settings, feature metadata. A sample ID yields the same vector everywhere.
Puneet + Samarth → Neelanshu	Fitted pipelines, validation predictions, run logs and feature metadata. Every prediction has a split and sample ID.
Neelanshu → Nakul + model owners	Verified tables, error figures, timing method and audit findings. Results trace back to saved predictions.
All four → Nakul	Their report sections, figure captions, references, contribution notes and slides.
Nakul → everyone	Integrated notebook, draft report and slides. Each member checks their section and practices the full demo.

Use these identifiers consistently

sample_id: original row number within the official partition, plus **source_split** = official_train or official_test. Train/validation IDs both refer to official_train. Never use the row number of a shuffled feature cache as the original sample ID.

Prediction CSV: run_id, stage, source_split, sample_id, y_true, y_pred. stage is val or test. Do not invent probabilities when a classifier only exposes decision scores. Preserve deterministic sample order.

Run record: run_id, owner, git_commit, seed, split_file_hash, train_n, val_n, feature_name, feature_dim, feature_config, scaler, estimator, hyperparameters, package_versions, fit_seconds, status, warnings and artifact_paths. Keep validation and test metrics in separate fields/files.

Results CSV: run_id, stage, train_n, feature, model, accuracy, macro_f1, fit_seconds, predict_ms_per_image, end_to_end_ms_per_image, serialized_bytes, complexity_type, complexity_value. Put per-class recall in a separate ten-row table per run. Use seconds, milliseconds and bytes explicitly.

Every handoff includes: what changed, how to run it, expected shapes, the evidence checked, and known limitations. The person receiving it must rerun a small example before accepting it.

14 / TEAM DELIVERY

Individual viva practice

Practice without reading the answers. Explain each idea simply, then open the relevant code or figure as evidence. These are suggested talking points, not answers to memorize word for word.

Everyone must answer these

What is the difference between validation and test?
Validation guides development. Test measures the frozen procedure on held-out examples. Looking at test failures and then tuning would contaminate that final assessment.

Why use both pixels and HOG?
They test whether a fixed description of local edges helps compared with raw intensities. HOG may help some classifiers and hurt others; the experiment must establish the result.

Why is accuracy alone insufficient?
It summarizes total correctness and can hide a weak category. Macro F1 averages class F1 values equally; per-class recall shows which categories are missed.

What can you honestly conclude about a CNN now?
Only that a CNN is a motivated Phase 2 direction. There is no evidence that your future CNN outperforms your ML baseline until that comparison is run.

Puneet: questions about your own work

Why can HOG have more features than pixels?
The proposed sliding blocks overlap. Six by six blocks each yield 36 values, so the descriptor length is 1,296; the goal is edge structure, not compression.

What happens when C decreases in LR?
Regularization becomes stronger. Whether validation performance improves depends on underfitting versus overfitting; it is a measured choice.

Why can kNN be costly with no learned weights?
It stores many training vectors and searches for nearby ones at prediction time. Report memory, feature dimension and prediction cost rather than calling it parameter-free computation.

Five-minute live demonstration

Show your own file and explain one nontrivial choice. Run a small cell or load a saved artifact. Trace one image from its original ID through features to a prediction. Read a validation metric correctly. Explain one failure, one limitation and one improvement reserved for Phase 2. Everyone must also describe the eight-row experiment and the shared split.

15 / TEAM DELIVERY

Suggested 10-day work plan

This is a relative plan, not an official deadline. Budget roughly 18-26 focused hours per person plus unattended compute; beginners may need longer. If you have fewer days, reduce the optional search grid and parallelize member-owned work. Keep data integrity, all eight baseline rows, and honest reporting.

Day	Your focus	Team gate
1	Learn fit/predict and features	All five understand scope and vocabulary
2	Inspect upright raw images	Environment works; literature notes started
3	Implement pixels and HOG	Data/split/feature contracts agreed
4	Smoke-test LR and kNN	One tiny run reaches the evaluator
5	Tune four feature/model rows	Protocol and compute budget fixed
6	Run frozen-size comparisons	Eight comparable rows progressing
7	Export models and predictions	Configurations frozen; test run authorized internally
8	Write feature/LR/kNN sections	Verified figures and individual text ready
9	Explain errors with Neelanshu	One full rehearsal; report compiles
10	Verify feature/model reproducibility	Fresh-run check; contribution log complete

Definition of done for every member

- Your files run in the agreed environment, your inputs and outputs are documented, and a teammate has reviewed one small run.
- Your report section explains what you did, why you did it, what evidence supports it, and what its limits are. Captions and citations are included.
- Your contribution log names real files, commits, experiments and reviews. It records actual work, not an artificial equal number of commits.
- You can explain your own implementation and the shared project during an individual viva. You have practiced a backup demo using saved outputs.

If a run fails, preserve the error and settings. Tell the team early and revise the compute plan consistently. Do not fabricate results, omit failed rows without explanation, or quietly change the split.

16 / TEAM DELIVERY

Sources and how to use them

Course source: CSAI_Project_Instructions_Phase_1_and_Phase_2.md, supplied by you. Its Phase 1 list and individual-viva rule govern the required work. Your pasted Project 11 description supplies the feature/model comparison and metrics. Neither source contains actual results for your team.

Technical references below were checked on 27 October 2026. Stable documentation can change; record the versions actually installed for your experiments. Reference IDs such as [R3] in this guide point to these links. Read and cite original material in your submitted report; this guide is not a substitute for a literature review.

[R1] Fashion-MNIST official repository: data files, label map and official partitions

[R2] Xiao, Rasul and Vollgraf (2017), Fashion-MNIST dataset paper

[R3] scikit-image: HOG API and settings

[R4] scikit-image: HOG worked explanation

[R5] scikit-learn: leakage and preprocessing pitfalls

[R6] scikit-learn: LogisticRegression

[R7] scikit-learn: KNeighborsClassifier

[R8] scikit-learn: RandomForestClassifier

[R9] scikit-learn: SVC

[R10] scikit-learn: model evaluation and metrics

[R11] scikit-learn: stratified train_test_split

[R12] Torchvision: FashionMNIST loader

[R13] Dalal and Triggs (2005): original HOG paper, CVPR

[R14] scikit-learn: LinearSVC alternative and multiclass behavior

[R15] LeCun et al. (1998): author-hosted document-recognition paper record

Suggested research reading beyond API pages

Read Dalal and Triggs (2005), "Histograms of Oriented Gradients for Human Detection" [R13], for the original descriptor. Its task is human detection, so do not transfer its scores to clothing classification. LeCun et al. (1998), "Gradient-Based Learning Applied to Document Recognition" [R15], is useful Phase 2 background; it does not require a Phase 1 CNN implementation.

Citation discipline

For each source record title, authors, year, task, representation, model, evaluation protocol and relevance. Separate a paper's results from your results. Repository leaderboard entries are not automatically controlled comparisons or verified targets. Use actual bibliographic metadata in your LaTeX .bib file; never cite an unread paper as evidence for a specific finding.

Code blocks are starter recipes to adapt and run in your project environment. They have not been run on Fashion-MNIST as part of creating these guides. Toy calculations are illustrative, not measured project performance.